

## B28: SQL SuperHero Program : Session 5

1 message

Vishal Jaiswal <vishaldbdev@gmail.com>  
Bcc: satishhattekar1997@gmail.com

17 February 2025 at 06:25

**Can we make ON DELETE CASCADE AND DEFERRABLE BOTH TOGETHER**

**Yes. Concept is same.**

```
create table tab1 (c varchar2(5) primary key);
create table tab2 (c varchar2(5) primary key);
```

```
alter table tab1 add constraint tab1_fk foreign key (c) references tab2(c) on delete cascade deferrable initially deferred
```

```
alter table tab2 add constraint tab2_fk foreign key (c) references tab1(c) on delete cascade deferrable initially deferred
```

Both created together

```
alter table tab1 add constraint tab1_fk foreign key (c) references tab2(c) on delete cascade deferrable initially deferred
```

Table altered.

```
alter table tab2 add constraint tab2_fk foreign key (c) references tab1(c) on delete cascade deferrable initially deferred
```

Table altered.

```
insert into tab1 values ('A');
insert into tab1 values ('B');
insert into tab1 values ('C');
insert into tab2 values ('A');
insert into tab2 values ('B');
insert into tab2 values ('C');
```

```
delete from tab1 where c = 'A'
```

1 row(s) deleted.

```
select * from tab2
```

|   |
|---|
| C |
| B |
| C |

2 rows selected.

Automatically deleted from tab2 also

Because ON DELETE CASCADE

**Question: What happen if we do not insert the record in both tables and try to commit the transaction?**

If we use the concept of **deferrable initially deferred** then , it will allow insert at that moment of time, but when we try to commit the transaction then it will go and check that must validate the FK rule. FK rule is saying that if data exists in child, it must exist in parent table. If you see the below example:

- We have tab1
- We have tab2
- Both tables have FK with **deferrable initially deferred**
- I have inserted 3 records in tab1 as 1,2,3
- I have inserted 2 record in tab2 as 1,2
- Now when we commit then, it wants to confirm that data in tab1 , must present in tab2 and data in tab2 must present in tab1
- Because "3" is only present in tab1, but not in tab2, so FK violates and all transaction rollback

```

Create table tab1 (c number Primary key);
Create table tab2 (c number Primary key);
alter table tab1 add constraint fk_tab1 foreign key (c) references tab2(c)
deferrable initially deferred;
alter table tab2 add constraint fk_tab2 foreign key (c) references tab1(c)
deferrable initially deferred;

Insert Into tab1 values (1);
Insert Into tab1 values (2);
Insert Into tab1 values (3);

Insert Into tab2 values (1);
Insert Into tab2 values (2);
  
```

*Handwritten notes:*  
 - Blue ink: ] - 3 Records in tab1  
 - Red ink: ] - 2 Records in tab2  
 - Green ink: Trying commit  
 - Green ink: At Run time it validate FK rule & rollback

**Question: Is it possible to create a self-referencing foreign key?**

**Answer: YES**

You create a self-referencing foreign key just like you do a regular foreign key, except that the referenced column (or columns) is on the same table where you're defining the foreign key. See the below example:

```

CREATE TABLE OurStuff
(
  StuffID Number PRIMARY KEY,
  StuffSubID Number,
  StuffName VARCHAR2(10),
  CONSTRAINT fk_StuffID FOREIGN KEY (StuffSubID)
  REFERENCES OurStuff(StuffID)
);

INSERT INTO OurStuff VALUES (1001, NULL, 'stuff1');
INSERT INTO OurStuff VALUES (1002, 1001, 'stuff2');
  
```

--> Till here no issue. Because the 2nd column (StuffSubID) which is a FK has the same value as exists in PK (StuffID). But if we try to insert any such records where StuffSubID has the value which is not exists in StuffID column, it gives same error --> parent key not found.

```
insert into ourstuff values (1003, 1004, 'fhdgi')
```

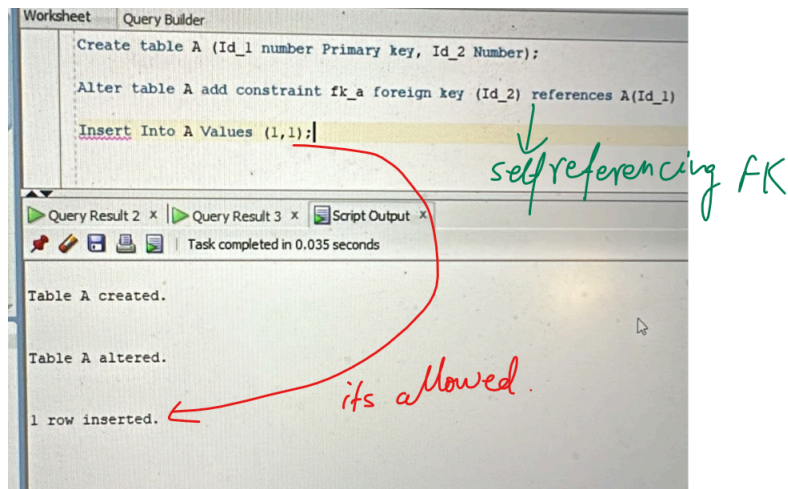
```
insert into ourstuff values (1003, 1004, 'fhdgj')
```

ORA-02291: integrity constraint (SQL\_BJOWVQUDSHBJCEYSZLJAGSBIE.FK\_STUFFID) violated - parent key not found 0

More Details: <https://docs.oracle.com/error-help/db/ora-02291>

--> Interviewer can ask any question here on the same concepts. Like employee and manager id in employee table. Every manager must be an employee, if we have to enforce this rule. we are creating self referencing FK

**Question: Can we create a self referencing FK and insert record 1,1, is it allowed?**



=====

**Question: Can I create a foreign key against only part of a composite primary key?**

**Answer: NO**

```
CREATE TABLE StuffType  
(  
   TypeID INT NOT NULL,  
    AltID INT NOT NULL,  
    TypeName VARCHAR(10) NOT NULL,  
    CONSTRAINT pk_StuffType PRIMARY KEY (TypeID, AltID)  
);
```

```
CREATE TABLE OurStuff  
(  
    StuffID INT NOT NULL PRIMARY KEY,  
    StuffName VARCHAR(10) NOT NULL,  
    OurTypeID INT NOT NULL,  
    CONSTRAINT fk_StuffType FOREIGN KEY (OurTypeID)  
    REFERENCES StuffType(TypeID)  
);
```

Here in the child table, I am trying to create FK on a part of PK. It gives errors as expected.

```
CREATE TABLE OurStuff
(
  StuffID INT NOT NULL PRIMARY KEY,
  StuffName VARCHAR(10) NOT NULL,
  OurTypeID INT NULL,
  CONSTRAINT fk_StuffType FOREIGN KEY (OurTypeID)
    REFERENCES StuffType(TypeID)
)
```

→ Part of PK of another table

ORA-02270: no matching unique or primary key for this column-list

--> **Why is this error coming?** Go to base concept that FK of one table refer to primary/unique key of another table. Here in the above example we created PK as a composite PK **PRIMARY KEY (TypeID, AltID)**, so while creating FK, it should refer to both as a composite, not as a part. but we are doing **REFERENCES StuffType(TypeID)**

**In the same scenario as above can 1 column from child table, refer to composite primary key**

No, Its not allowed. Number of columnn should match

```
create table abcd(
stuffid number primary key,
stuffname varchar2(10),
ourtypeid number,
outaltid number,
constraint fk_abcd foreign key (outaltid) references xyz(typeid,altid)
)
```

1 column

2 columns  
as  
Composite  
PK

ORA-02256: number of referencing columns must match referenced columns

```

1  create table xyz(
2  typeid number not null,
3  altid number unique,
4  typename varchar2(20),
5  constraint pk_xyz primary key (typeid,altid)
6  );
7
8
9
10 create table abcd(
11 stuffid number primary key,
12 stuffname varchar2(10),
13 ourtypeid number,
14 constraint fk_abcd foreign key (ourtypeid) references xyz(altid)
15 )|

```

Table created.

allowed

See the above scenario as we discussed in session. Here we have composite PK, and altid is an attribute of composite PK, but it is alone defined as a Unique key. The concept of FK is : It refers to PK or UK. So here because it is UK then we can refer.

Let's go back on previous question. Can I create a part of PK, so we can not. so how to do that :

```

CREATE TABLE OurStuff
(
  StuffID INT NOT NULL PRIMARY KEY,
  StuffName VARCHAR(10) NOT NULL,
  OurTypeID INT NULL,
  ouratlid number,
  CONSTRAINT fk_StuffType FOREIGN KEY (OurTypeID,ouratlid)
  REFERENCES StuffType(TypeID, altid)
)

```

Table created.

2 column  
(The composite PK from Parent)

2 column from child

we can do

Can we create FK on the same column which is PK already?

YES

```
CREATE TABLE supplier
( supplier_id number(10) primary key,
  supplier_name varchar2(50) not null,
  contact_name varchar2(50)
)
```

→ will make it  
Parent

Table created.

```
CREATE TABLE products
( product_id number(10) not null,
  supplier_id number(10) primary key
)
```

→ will make  
this  
child

Table created.

```
alter table products add constraint fk_vishal foreign key (supplier_id) references supplier (supplier_id)
```

↓ This is PK & we created  
FK as  
way

Table altered.

✓ its done

But just be aware that this time it has PK + FK both. This can not accept NULL, because it is PK as well. It will not accept duplicates as well, even from FK side there is no as such rule for duplicate data, but this column is PK as well, so it must follow those characteristics.

When To Use Surrogate Key and When to Use PK?

surrogate keys are meaningless. They serve no purpose, except to technically identify a record uniquely in one source system. It is manufactured “artificially” and only for the purposes of data analysis.

When we do not want to use composite PK, we decide to use SK.

The primary key is a unique key in your table that you choose that best uniquely identifies a record in the table.

A surrogate key is an artificially generated key. They're useful when your records essentially have no natural key

Top Reasons to Always use Surrogate Keys in Data Warehousing:

- surrogate keys will save index space and improve performance.
- Slow Changing Dimensions.(Data Is changing frequently) It is often a requirement to track historical values of dimension records
- Surrogate keys are unique, Because surrogate keys are system-generated, it is impossible for the system to create and store a duplicate value
- The value is never reused and is unique within the whole system.
- It is system generated.
- The value cannot be manipulated by the user or application.

**A surrogate key** is a made up value with the sole purpose of uniquely identifying a row. Usually, this is represented by an auto incrementing ID. The word surrogate means substitute. A surrogate key is an attribute that is invented. An example of a surrogate key is an “address ID” for a table of addresses. Outside of the system, an address ID has no value to anyone. But for the database, it's used to uniquely identify the record.

In other words, it is a column added to the table. We can use a surrogate key when there are no columns to define as a natural primary key. These keys are unique as we create it when there is no other natural primary key.

**Constraints: (when ever we have to strict user over the database level, we have to use constraints)**

## Constraints

Oracle constraints are critical to the scalability, flexibility and integrity of your database data. Constraints apply specific rules to data, ensuring the data conforms to the requirements defined. There are a number of different kinds of constraints that you will be concerned with as a DBA. These are:

1. Check
2. Not NULL
3. Primary key
4. Unique
5. Foreign Key
6. Some websites are talking about Default constraints. Default constraint is a rule defined on a column that specifies a default value for that column when a new row is inserted into the table without explicitly specifying a value for that column. It means, when we add a new column and specify a default values (check notes 2 or 3 -- add a new column and we added a default value on it)

```
create table my_status (
  status_id number primary key,
  person_id number not null,
  ssn varchar2(20) unique,
  active_record varchar2(1)
)
```

```
insert into my_status values (1, null, 'sgsgs', 'T')
```

--> **NOT NULL** constrains strict users that we can not insert NULL values in that column.

The screenshot shows an Oracle SQL error message: "ORA-01400: cannot insert NULL into ('SQL\_CJVZPSPOUGGLOYFAFDIVRFMMB'. 'MY\_STATUS'. 'PERSON\_ID') (". Handwritten red notes include: "stop you fo insert null here" with an arrow pointing to the error message, and "blow q NOT NULL constraint" with an arrow pointing to the column name 'PERSON\_ID' in the error message.

The screenshot shows a SQL query result: "select \* from my\_status". The result is a table with four columns: STATUS\_ID, PERSON\_ID, SSN, and ACTIVE\_RECORD. The first row has values 1, 2, sgsgs, and T. The second row has values 2, 2, sgttsgs, and T. Below the table, it says "2 rows selected.". Handwritten red notes include: "duplicate possible (null not possible)" with an arrow pointing to the second row.

| STATUS_ID | PERSON_ID | SSN     | ACTIVE_RECORD |
|-----------|-----------|---------|---------------|
| 1         | 2         | sgsgs   | T             |
| 2         | 2         | sgttsgs | T             |

Lets Create a CHECK Constraint :

```
alter table my_status add constraint ck_status check(active_record = 'T' OR active_record = 'F') ;
```

This above constraint, will check each data in active\_record column, it must be T or F only.

insert into my\_status values (3, 3, 'retwrtr', 'X'); --> This will give you a check constraint violated error, because we created a check constraint on column active\_record and it should accept the values "T" and "F" only. here we are trying to insert "X" which is not acceptable.

```
insert into my_status values (3, 3, 'retwrtr', 'X')
```

→ not allowed

ORA-02290: check constraint (SQL\_CJVZPSPOUGGLOYFAFDIVRFMMB.CK\_STATUS) violated ORA-06512: at "S

More Details: <https://docs.oracle.com/error-help/db/ora-02290>

```
insert into my_status values (3, 3, 'retwrtr', 'F')
```

↳ allowed

1 row(s) inserted.

Example, you can explain in interview, that only age > 18 allowed in factory

```
create table test123(  
id number,  
age number  
)
```

```
ALTER TABLE test123  
ADD (CONSTRAINT ck_test123  
CHECK ( age > 18));
```

```
insert into test123 values (1,10);
```

not allowing value 10 in the age column, because we created a check constraint that always checks the value before insertion and it must be > 18 .

```
insert into test123 values (1, 21)
```

→ > 18 allowed

1 row(s) inserted.

```
insert into test123 values (2, 17)
```

→ < 18 not allowed

ORA-02290: check constraint (SQL\_CJVZPSPOUGGLOYFAFDIVRFMMB.CK\_TEST) violated ORA-0651

More Details: <https://docs.oracle.com/error-help/db/ora-02290>

We can create Check constraints on any datatype. Whenever I want to check that only these values are allowed, then we will use CHECK constraint.

```
create table test123 (id number, date_of_birth date)
```

→ Table created

Table created.

```
alter table test123 add (constraint nitin check(date_of_birth > '20-DEC-2020'))
```

Table altered.

← Constraint created

```
insert into test123 values (2, '01-JAN-2021')
```

→ This is OK, as date-of-birth >

1 row(s) inserted.

```
insert into test123 values (3, '01-JAN-2020')
```

→ Not allowed.

ORA-02290: check constraint (SQL\_IQEGQSHAMMBFFKRUSIXQEIOH.NITIN) violated ORA-06512: at "SYS.DBMS\_SQL", line 1721

Question:

How to ensure a constraint to check **Answer column of Questions table** allows only 'A','B','C','D','P');

Answer:

We have to create a check constraint, which will validate the record before insert.

**ALTER TABLE Questions**

**ADD (CONSTRAINT ck\_Answer**

**CHECK (Answer ='A' or**

**Answer='B' or**

**Answer='C' or**

**Answer='D' or**

**Answer='P'**

**)**

**);**

After this if we try to insert a record like below:

```
Insert into Questions values ('Q1','frthy','ju','loi','hytu','jkui','A',sysdate);
```

This will inserted, because values in Answer column is A, but if try to insert any other value other than 'A','B','C','D','P' Then it will not allow:

```
Insert into Questions values ('Q1','frthy','ju','loi','hytu','jkui','X',sysdate);
```

Here I am trying to insert a value 'X' in answer column, it gives error.

02290. 00000 - "check constraint (%s.%s) violated"

\*Cause: The values being inserted do not satisfy the named check

=====

**We can do multiple check on CHECK Constraint as well.**

```
create table products (  
    product_id number primary key,  
    product_name varchar2(20),  
    price number,  
    discount number,  
    check (price > 0 and discount >=0 and discount <= 100)  
);
```

The screenshot shows a SQL command and its resulting error message. The command is: `insert into products values (2, 'laptop1', 203, 200)`. The error message is: `ORA-02290: check constraint (SQL_CJVZPSPOUGGLOYFAFDIVRFMMB.SYS_C00172225068)`. Handwritten annotations in purple and red ink are present. A purple line points from the word "satisfied" to the command. A red arrow points from the word "Violated" to the error message. Another red arrow points from the text "not allow" to the error message. At the bottom, red text says "Both must satisfy".

satisfied

Violated

not allow

Both must satisfy

**Question: what is the constraint\_type for null constraint?**

In User\_Constraints table, we can check via constraint\_type = 'C'. Null constraint is also a type of check constraint as well. It is checking that value should not be null.

-----

Question: can we insert Null value in check constraint.

YES.  
Again same concepts, check constraint will enforce the rule that only these values are allowed. So, it will use comparison and null can not be compare , so database will unable to verify null records will the values which are allowed. To stop null values in check constraint column, we should create check constraint with NOT NULL

Can NOT Null constraint allow duplicate ?

YES, it is just NOT NULL. Means, null not allwed.

Question: Can we disable constraints ?

```
Create table my_status
(status_id NUMBER PRIMARY KEY,
person_id NUMBER NOT NULL,
active_record VARCHAR2(1) NOT NULL,
person_ssn VARCHAR2(20) CONSTRAINT un_person_ssn UNIQUE
);
```

```
ALTER TABLE my_status
ADD (CONSTRAINT ck_vishal
CHECK (UPPER(active_record)='T' or
UPPER(active_record)='F'));
```

```
insert into my_status values (1,1,'T','22');
```

→ Because of check constraint

```
insert into my_status values (2,3,'X',334)
```

ORA-02290: check constraint (SQL\_YKQKOWTHZWFNDJXOPOQAFFHT.CK\_VISHAL) violated ORA-06512: at

Lets disable the constraint and see

```
15 alter table my_status disable constraint ck_vishal;
16
17 insert into my_status values (3,1,'X','26');
18
19 select * from my_status;
20
```

disabled

Inserted

| STATUS_ID | PERSON_ID | ACTIVE_RECORD | PERSON_SSN |
|-----------|-----------|---------------|------------|
| 1         | 1         | T             | 22         |
| 2         | 1         | X             | 21         |
| 3         | 1         | X             | 26         |

Download CSV  
3 rows selected.

Added

We can enable it again with novalidate option as Foreign key.

```
alter table my_status enable novalidate constraint ck_vishal;
```

```
alter table my_status enable constraint ck_vishal
```

] - enable not working, because we have data which not satisfy

```
ORA-02293: cannot validate (SQL_YKQKOWTHZWFNDJXOPOQAFFHT.CK_VISHAL) - check constraint violated
```

```
alter table my_status enable novalidate constraint ck_vishal
```

] - enable with novalidate

```
Table altered.
```

```
insert into my_status values (4,3,'Y',444)
```

] → Now start working

```
ORA-02290: check constraint (SQL_YKQKOWTHZWFNDJXOPOQAFFHT.CK_VISHAL) violated ORA-06512: at "SYS.DBMS_SQL", line 1721
```